Zuul Project Report: Pokémon Delta Emerald

Site Web: https://perso.esiee.fr/~jouanarb/ (a venir du a des problèmes d'accès)

https://clascheurduturfu.github.io/Barnabe-Jouanard/

Groupe: A3P 2025/2026 G5

I.A) Auteur : Barnabe Jouanard

I.B) Thème

Un jeu d'aventure se déroulant dans la région de Hoenn (Pokémon), axé sur l'exploration et la résolution d'énigmes environnementales pour mettre fin à un conflit légendaire entre la terre et la mer.

I.C) Résumé du Scénario

Le joueur incarne un dresseur à Littleroot Town. Le monde subit un climat anormal. L'objectif principal est de traverser la partie ouest de Hoenn pour trouver l'Orbe Émeraude (cachée à Petalburg City) et l'apporter au Pilier Céleste pour réveiller Rayquaza.

I.D) Full Plan

La carte se compose de 7 lieux principaux reliés ainsi :

Your House: Starting location.

Littleroot Town: Connecting hub.

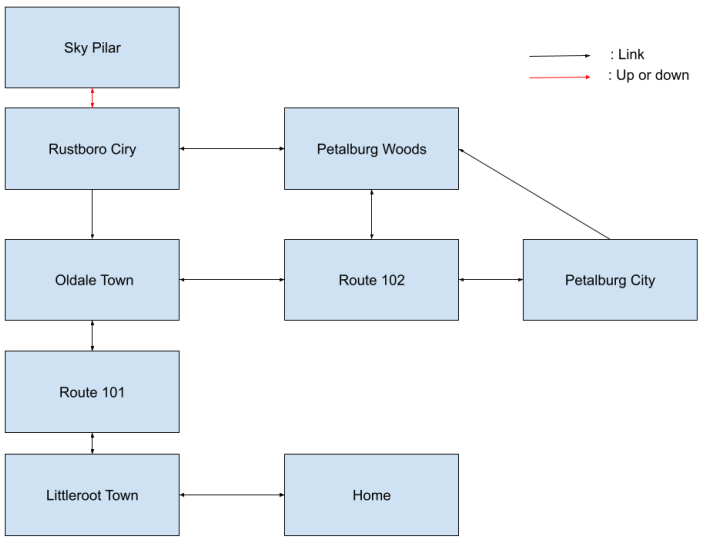
Route 101: Path to the north.

Oldale Town: Junction to the west.

Route 102: Combat/Trainer route.

Petalburg City: City containing a locked Gym.

Sky Pillar: The final destination.



I.E) Scenario Detaille

Le joueur commence dans sa Maison. Il doit se rendre à Bourg-en-Vol puis sur la Route 101 pour rencontrer le Professeur Seko. Après avoir reçu des instructions, il se dirige vers Bourg-en-

Paume et Clémenti-Ville. Le joueur devra trouver un chemin à travers les Bois Clémenti pour atteindre le Pilier Céleste.

I.F) Details of Locations, Items, and Characters

Personnages : Professeur Seko (donne des indices), Norman (Champion d'arène à Rustboro City).

Objets : Chaussures de Sport : (Futur) pour se déplacer plus vite.

Orbe Émeraude : Essentiel pour invoquer Rayquaza.

I.G) Conditions de Victoire/Défaite

Victoire : Atteindre le sommet du Pilier Céleste en possédant l'Orbe Émeraude.

Défaite : Un compteur de tours simulant l'aggravation de la météo. Si le compteur atteint zéro, la région est détruite.

I.I) Comments

La version 1 n'a pour l'instant que très peu des fonctionnalités prévues et est une base pour la suite du projet.

II. Réponses aux Exercices

Exercice 7.5

J'ai implémenté la méthode privée printLocationInfo() dans Game pour centraliser l'affichage de la description de la pièce et des sorties. Cela évite la duplication de code entre printWelcome() et goRoom().

```
private void printLocationInfo()
{
    System.out.println("You are " + this.aCurrentRoom.getDescription());
    System.out.print("Exits : ");
    if (this.aCurrentRoom.aNorthExit != null){
        System.out.print("North ");
    }
    if (this.aCurrentRoom.aSouthExit != null){
        System.out.print("South ");
    }
    if (this.aCurrentRoom.aEastExit != null){
        System.out.print("East ");
    }
    if (this.aCurrentRoom.aWestExit != null){
        System.out.print("West ");
    }
}
```

Exercice 7.6 :

J'ai passé les attributs de la classe Room en private et créé l'accesseur getExit pour masquer les détails internes des sorties. Cette modification réduit le couplage en permettant à la classe Game de demander une direction sans accéder directement aux variables de Room.

```
Room vNextRoom = this.aCurrentRoom.getExit(vDirection);
```

```
public Room getExit(final String pDirection){
    if (pDirection.equals("North"))
    {
        return aNorthExit;
    }
    if (pDirection.equals("South"))
    {
        return aSouthExit;
    }
    if (pDirection.equals("East"))
    {
        return aEastExit;
    }
    if (pDirection.equals("West"))
    {
        return aWestExit;
    }
    return null;
}
```

Exercice 7.7 :

J'ai créé la méthode getExitString dans Room pour déléguer la préparation de la liste des sorties à la pièce elle-même. Cela permet à Game d'afficher les informations sans avoir à connaître la structure interne des sorties de Room.

```
public String getExitString()
{
    String vReturnString = "Exits:";

    if (this.getExit("North") != null) {
    vReturnString += " North";
    }

    if (this.getExit("East") != null) {
    vReturnString += " East";
    }

    if (this.getExit("South") != null) {
    vReturnString += " South";
    }

    if (this.getExit("West") != null) {
    vReturnString += " West";
    }

    return vReturnString;
} // getExitString()
```

```java
private void printLocationInfo()
{
    System.out.println("You are " + this.aCurrentRoom.getDescription());
    System.out.print(this.aCurrentRoom.getExitString());
    System.out.println();
}
```

Exercice 7.8 :

J'ai remplacé les attributs de direction par une HashMap<String, Room> pour centraliser la gestion des sorties. J'ai adapté getExitString, printLocationInfo, getExit et le constructeur pour HashMap et aussi réécrit dans la classe game les sorties pour adapter ces changements.

```java
private HashMap<String, Room> aExits;
```

```java
public Room (final String pDescription)
{
    this.aDescription = pDescription;
    this.aExits = new HashMap<String, Room>();
} // Room()
```

```java
public Room getExit(final String pDirection)
{
    return this.aExits.get(pDirection);
} // getExit()
```

```java
public String getExitString()
{
    String vReturnString = "Exits:";

    if (this.aExits.get("North") != null) {
    vReturnString += " North";
    }

    if (this.aExits.get("East") != null) {
    vReturnString += " East";
    }

    if (this.aExits.get("South") != null) {
    vReturnString += " South";
    }

    if (this.aExits.get("West") != null) {
    vReturnString += " West";
    }
```

```java
public void setExit (final String pDirection,final Room pExit)
{
    this.aExits.put(pDirection, pExit);
} // setExits()
```

```java
    vHouse.setExit("West", vLittleroot);

    vLittleroot.setExit("North", vRoute101);
    vLittleroot.setExit("East", vHouse);

    vRoute101.setExit("North", vOldale);
    vRoute101.setExit("South", vLittleroot);

    vOldale.setExit("East", vRoute102);
    vOldale.setExit("South", vRoute101);

    vRoute102.setExit("North", vPetalburgWoods);
    vRoute102.setExit("East", vPetalburg);
    vRoute102.setExit("West", vOldale);

    vPetalburg.setExit("South", vRoute102);
    vPetalburg.setExit("West", vRustboro);

    vPetalburgWoods.setExit("South", vRoute102);
    vPetalburgWoods.setExit("West", vRustboro);
```

Exercice 7.8.1 :

J'ai étendu la gestion des déplacements en ajoutant les directions verticales "up" et "down", permettant ainsi d'accéder au Sky Pillar. Pour cela, j'ai simplement utilisé la nouvelle méthode setExit pour lier les pièces et mis à jour getExitString pour afficher ces sorties supplémentaires ainsi que goRoom pour pouvoir se déplacer.

```java
public String getExitString()
{
    String vReturnString = "Exits:";

    if (this.aExits.get("North") != null) {
    vReturnString += " North";
    }

    if (this.aExits.get("East") != null) {
    vReturnString += " East";
    }

    if (this.aExits.get("South") != null) {
    vReturnString += " South";
    }

    if (this.aExits.get("West") != null) {
    vReturnString += " West";
    }

    if (this.aExits.get("Down") != null) {
    vReturnString += " Down";
    }

    if (this.aExits.get("Up") != null) {
    vReturnString += " Up";
    }

    return vReturnString;
} // getExitString()
```

```java
        else if (pCommand.getSecondWord().equals("Up"))
        {
            if (this.aCurrentRoom.getExit("Up")!=null)
            {
                this.aCurrentRoom=this.aCurrentRoom.getExit("Up");
                this.printLocationInfo();
            } else
            {
                System.out.println("There is no door !");
            }
        }
        else if (pCommand.getSecondWord().equals("Down"))
        {
            if (this.aCurrentRoom.getExit("Down")!=null)
            {
                this.aCurrentRoom=this.aCurrentRoom.getExit("Down");
                this.printLocationInfo();
            } else
            {
                System.out.println("There is no door !");
            }
        }
```

```java
    vRustboro.setExit("Up", vSkyPillar);
    vRustboro.setExit("East", vPetalburgWoods);

    vSkyPillar.setExit("Down", vRustboro);
```

Exercice 7.9 :

J'ai remplacé la vérification manuelle des directions dans getExitString par une boucle for-each parcourant l'ensemble des clés (keySet) de la HashMap . Cette modification permet de construire dynamiquement la liste des sorties disponibles, rendant la classe Room capable d'afficher n'importe quelle nouvelle direction (comme "up" ou "down") sans aucune modification de code supplémentaire.

```java
public String getExitString()
{
    String vReturnString = "Exits:";
    Set<String> vKeys = this.aExits.keySet(); // Récupère toutes les directions présentes
    for (String vExit : vKeys) {
        vReturnString += " " + vExit;
    } // for
    return vReturnString;
} // getExitString()
```

Exercice 7.11 :

J'ai implémenté la méthode getLongDescription() dans la classe Room afin qu'elle assemble elle-même la description du lieu et la liste des sorties. La méthode printLocationInfo() de la classe Game a été simplifiée pour n'appeler que cette fonction, masquant ainsi totalement les détails d'implémentation interne de la pièce au reste de l'application .

```java
public String getLongDescription()
{
    return "You are " + this.getDescription() + '\n' + this.getExitString();
}
```

```java
private void printLocationInfo()
{
    System.out.println("You are " + this.aCurrentRoom.getLongDescription());
} // printLocationInfo()
```

Exercice 7.14 :

J'ai créé la commande look qui permet au joueur d'afficher la description complète du lieu actuel et de ses sorties sans effectuer de déplacement. Pour améliorer l'expérience utilisateur, j'ai également mis à jour la liste des mots de commande dans CommandWords afin que look apparaisse lors de l'appel à la commande help.

```java
private final String[] aValidCommands = { "go", "help", "quit", "look" };
```

```java
    else if (pCommand.getCommandWord().equals("look")){
        look();
        return false;
```

```java
private void look()
{
    System.out.println(this.aCurrentRoom.getLongDescription());
} //look()
```

```java
private void printHelp()
{
    System.out.println("You are lost. You are alone.");
    System.out.println("You wander around at the university.");
    System.out.println("Your command words are:");
    System.out.println("  go quit help look");
} // printHelp()
```

Exercice 7.15 :

J'ai ajouté la commande eat qui, une fois activée, affiche un message indiquant que le joueur a mangé et n'a plus faim. Techniquement, cette commande a été enregistrée dans la classe CommandWords et intégrée à la structure de contrôle de la méthode processCommand dans la classe Game.

```java
private final String[] aValidCommands = { "go", "help", "quit", "look", "eat" };
```

```java
    else if (pCommand.getCommandWord().equals("eat")){
        eat();
        return false;
```

```java
private void eat()
{
    System.out.println("You have eaten now and you are not hungry any more.");
} //eat()
```

```java
private void printHelp()
{
    System.out.println("You are lost. You are alone.");
    System.out.println("You wander around at the university.");
    System.out.println("Your command words are:");
    System.out.println("  go quit help look eat");
} // printHelp()
```

Exercice 7.16 :

J'ai délégué la responsabilité de l'affichage des commandes à la classe CommandWords via une méthode showAll(), appelée par le Parser. Ce changement permet à la commande help de mettre à jour automatiquement sa liste de mots (incluant look et eat) sans qu'aucune modification ne soit nécessaire dans la classe Game.

```java
public void showAll(){
    for(String vCommand : aValidCommands){
        System.out.print(vCommand + ' ');
    }
    System.out.println();
} // showAll()
```

```java
public void showCommands(){
    this.aValidCW.showAll();
} // showCommands()
```

```java
private void printHelp()
{
    System.out.println("You are in the wonderful word of Pokemon.");
    System.out.println("You are trying to stop Rayquazza from destroying Hoenn.");
    System.out.println("Your command words are:");
    aParser.showCommands();
} // printHelp()
```

III. Guide Utilisateur

Pour jouer au jeux:

Créer une instance de Game.

Appeler la méthode play().

Utiliser les commandes : go [direction](north; east; south; west; up; down) , look, eat, help, quit.

IV. Déclaration Anti-Plagiat

Moi, Barnabé Jouanard, déclare n'avoir copié aucune ligne de code d'une source externe